Lab Assignment-1 — House Prices

SVNIT | Department of Artificial Intelligence | Machine Learning (AI301)

Question sheet: `Labassignment-1.pdf`
Dataset: Kaggle House Prices → `../data/train.csv`

Part	Topic
A	Data loading & basic inspection
B	Univariate analysis
C	Bivariate analysis
D	Missing value treatment
E	Outlier detection & treatment
F	Feature encoding & scaling
G	Linear regression

In [1]:

```
import warnings
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from scipy import stats
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, Standar
dScaler

warnings.filterwarnings("ignore")
%matplotlib inline
pd.set_option("display.float_format", lambda x: f"{x:,.4f}")

ROOT = Path("..").resolve()
DATA = ROOT / "data"
OUTPUTS = ROOT / "outputs"
PLOTS = ROOT / "plots" / "labassignment_1"
OUTPUTS.mkdir(parents=True, exist_ok=True)
PLOTS.mkdir(parents=True, exist_ok=True)


def section(title):
    print("\n" + "=" * 64)
    print(f"  {title}")
    print("=" * 64)


def pct(x, digits=2):
    return f"{100 * x:.{digits}f}%"


def money(x):
    return f"${x:,.2f}"


def show_plot(name):
    path = PLOTS / name
    plt.tight_layout()
    plt.savefig(path, dpi=120, bbox_inches="tight")
    plt.show()
    print(f"[saved] {path.relative_to(ROOT)}")


print(f"Project root: {ROOT}")
```

```
Project root: /Users/ojas/Desktop/ML Lab
```

Part A: Data Loading & Basic Inspection

In [2]:
```
section("Part A: Data Loading & Basic Inspection")

df = pd.read_csv(DATA / "train.csv")

print("First 10 rows:")
display(df.head(10))

print(f"\nRows:    {df.shape[0]:,}")
print(f"Columns: {df.shape[1]}")

numerical_cols = df.select_dtypes(include=[np.number]).columns.tolist
()
categorical_cols = df.select_dtypes(include=["object"]).columns.tolist
()
print(f"\nNumerical columns:   {len(numerical_cols)}")
print(f"Categorical columns: {len(categorical_cols)}")

print("\nDatatype summary:")
print(df.dtypes.value_counts().to_string())

print("\nIncorrect datatype check:")
print("  MSSubClass is stored as int but represents house class catego
ries.")
print("  → Convert MSSubClass to string (categorical).")
df["MSSubClass"] = df["MSSubClass"].astype(str)

numerical_cols = df.select_dtypes(include=[np.number]).columns.tolist
()
categorical_cols = df.select_dtypes(include=["object"]).columns.tolist
()
print(f"\nAfter conversion → Numerical: {len(numerical_cols)} | Catego
rical: {len(categorical_cols)}")
```

```
============================================================
  Part A: Data Loading & Basic Inspection
============================================================
First 10 rows:
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Uti
0	1	60	RL	65.0000	8450	Pave	NaN	Reg	Lvl	A
1	2	20	RL	80.0000	9600	Pave	NaN	Reg	Lvl	A
2	3	60	RL	68.0000	11250	Pave	NaN	IR1	Lvl	A
3	4	70	RL	60.0000	9550	Pave	NaN	IR1	Lvl	A
4	5	60	RL	84.0000	14260	Pave	NaN	IR1	Lvl	A
5	6	50	RL	85.0000	14115	Pave	NaN	IR1	Lvl	A
6	7	20	RL	75.0000	10084	Pave	NaN	Reg	Lvl	A
7	8	60	RL	NaN	10382	Pave	NaN	IR1	Lvl	A
8	9	50	RM	51.0000	6120	Pave	NaN	Reg	Lvl	A
9	10	190	RL	50.0000	7420	Pave	NaN	Reg	Lvl	A

10 rows × 81 columns

```
Rows:    1,460
Columns: 81

Numerical columns:   38
Categorical columns: 43

Datatype summary:
str        43
int64      35
float64     3

Incorrect datatype check:
  MSSubClass is stored as int but represents house class categories.
  → Convert MSSubClass to string (categorical).

After conversion → Numerical: 37 | Categorical: 44
```

Part B: Univariate Analysis

In [3]:

```
section("Part B: Numerical features")

key_num = ["LotArea", "GrLivArea", "SalePrice", "YearBuilt", "OverallQ
ual"]

fig, axes = plt.subplots(2, 3, figsize=(14, 8))
for i, col in enumerate(key_num):
    sns.histplot(df[col].dropna(), kde=True, ax=axes.flatten()[i], col
or="steelblue")
    axes.flatten()[i].set_title(col)
axes.flatten()[-1].axis("off")
show_plot("part_b_histograms.png")

print(f"\n{'Feature':<14} {'Skewness':>10}  Shape")
print("-" * 44)
for col in key_num:
    s = df[col].skew()
    shape = "approximately normal" if abs(s) < 0.5 else ("right-skewe
d" if s > 0 else "left-skewed")
    print(f"{col:<14} {s:>10.3f}  {shape}")

section("Part B: Categorical features")

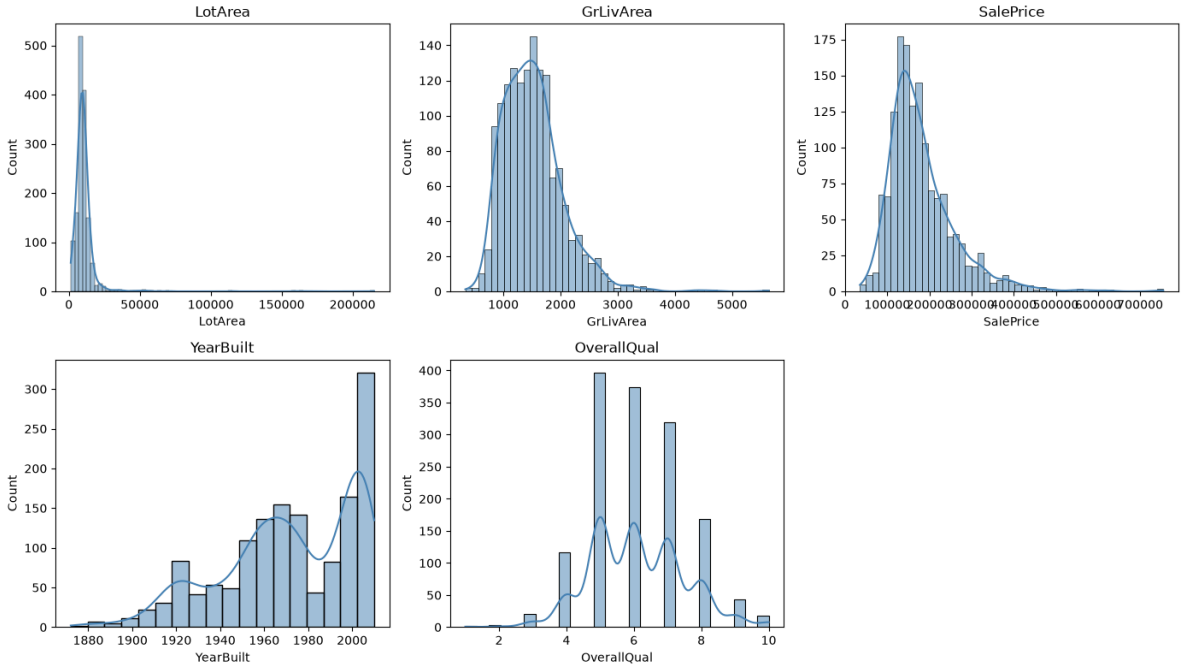
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for ax, col in zip(axes, ["Neighborhood", "HouseStyle", "BldgType"]):
    counts = df[col].value_counts()
    sns.barplot(x=counts.index, y=counts.values, ax=ax, color="steelbl
ue")
    ax.set_title(col)
    ax.tick_params(axis="x", rotation=45)
    print(f"\n{col}")
    print(f"  Most common: {counts.idxmax()} ({counts.max()} rows, {pc
t(counts.max()/len(df))})")
    print(f"  Rarest:      {counts.idxmin()} ({counts.min()} rows, {pc
t(counts.min()/len(df))})")
show_plot("part_b_barcharts.png")

rows = []
for col in categorical_cols:
    for cat, n in df[col].value_counts().items():
        rows.append({"Column": col, "Category": cat, "Count": n, "% of
rows": round(100 * n / len(df), 2)})
top10 = pd.DataFrame(rows).sort_values("Count", ascending=False).head
(10).reset_index(drop=True)
print("\nTop 10 most frequent categories across all categorical featur
es:")
print(top10.to_string(index=False))

print("\nImbalanced categories (dominant value > 80%):")
print(f"{'Column':<16} {'Dominant':<22} {'Share'}")
print("-" * 48)
for col in categorical_cols:
```

```
    vc = df[col].value_counts(normalize=True, dropna=False)
    if vc.iloc[0] > 0.80:
        print(f"{col:<16} {str(vc.index[0]):<22} {pct(vc.iloc[0])}")
```

```
============================================================
  Part B: Numerical features
============================================================
```




```
[saved] plots/labassignment_1/part_b_histograms.png

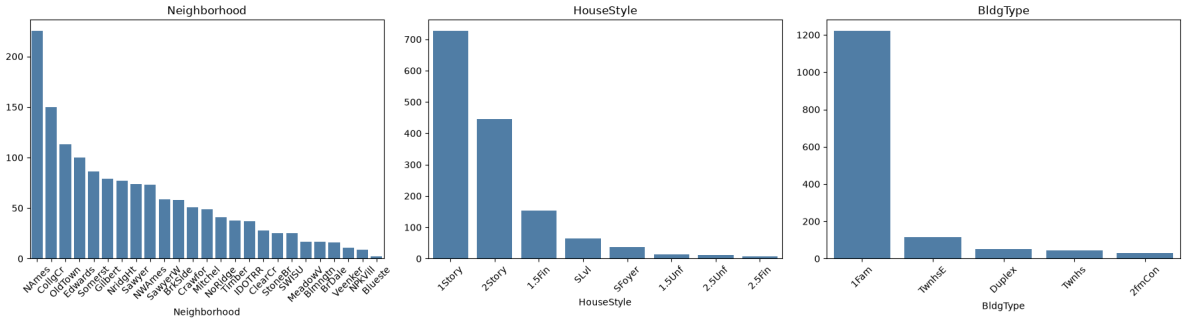
Feature          Skewness  Shape
------------------------------------------
LotArea            12.208  right-skewed
GrLivArea           1.367  right-skewed
SalePrice           1.883  right-skewed
YearBuilt          -0.613  left-skewed
OverallQual         0.217  approximately normal

============================================================
  Part B: Categorical features
============================================================

Neighborhood
  Most common: NAmes (225 rows, 15.41%)
  Rarest:      Blueste (2 rows, 0.14%)

HouseStyle
  Most common: 1Story (726 rows, 49.73%)
  Rarest:      2.5Fin (8 rows, 0.55%)

BldgType
  Most common: 1Fam (1220 rows, 83.56%)
  Rarest:      2fmCon (31 rows, 2.12%)
```



[saved] plots/labassignment_1/part_b_barcharts.png

Top 10 most frequent categories across all categorical features:

Column	Category	Count	% of rows
Utilities	AllPub	1459	99.9300
Street	Pave	1454	99.5900
Condition2	Norm	1445	98.9700
RoofMatl	CompShg	1434	98.2200
Heating	GasA	1428	97.8100
LandSlope	Gtl	1382	94.6600
CentralAir	Y	1365	93.4900
Functional	Typ	1360	93.1500
PavedDrive	Y	1340	91.7800
Electrical	SBrkr	1334	91.3700

Imbalanced categories (dominant value > 80%):

Column	Dominant	Share
Street	Pave	99.59%
Alley	nan	93.77%
LandContour	Lvl	89.79%
Utilities	AllPub	99.93%
LandSlope	Gtl	94.66%
Condition1	Norm	86.30%
Condition2	Norm	98.97%
BldgType	1Fam	83.56%
RoofMatl	CompShg	98.22%
ExterCond	TA	87.81%
BsmtCond	TA	89.79%
BsmtFinType2	Unf	86.03%
Heating	GasA	97.81%
CentralAir	Y	93.49%
Electrical	SBrkr	91.37%
Functional	Typ	93.15%
GarageQual	TA	89.79%
GarageCond	TA	90.82%
PavedDrive	Y	91.78%
PoolQC	nan	99.52%
Fence	nan	80.75%
MiscFeature	nan	96.30%
SaleType	WD	86.78%
SaleCondition	Normal	82.05%

Part C: Bivariate Analysis

In [4]:

```
section("Part C: Bivariate Analysis")

plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x="GrLivArea", y="SalePrice", alpha=0.55)
plt.title("GrLivArea vs SalePrice")
show_plot("part_c_scatter.png")
print("Trend: positive — larger GrLivArea generally means higher SaleP
rice.")

plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x="OverallQual", y="SalePrice")
plt.title("SalePrice by OverallQual")
show_plot("part_c_boxplot_overallqual.png")

numeric_df = df.select_dtypes(include=[np.number])
corr_sale = numeric_df.corr()["SalePrice"].drop("SalePrice")
top10_feats = corr_sale.abs().sort_values(ascending=False).head(10)

print("\nTop 10 features correlated with SalePrice:")
print(f"{'Feature':<16} {'r':>10}  |r| %")
print("-" * 36)
for feat in top10_feats.index:
    r = corr_sale[feat]
    print(f"{feat:<16} {r:>+10.3f}  {pct(abs(r))}")

plt.figure(figsize=(10, 8))
sns.heatmap(numeric_df[list(top10_feats.index) + ["SalePrice"]].corr
(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Top 10 Correlations with SalePrice")
show_plot("part_c_heatmap.png")

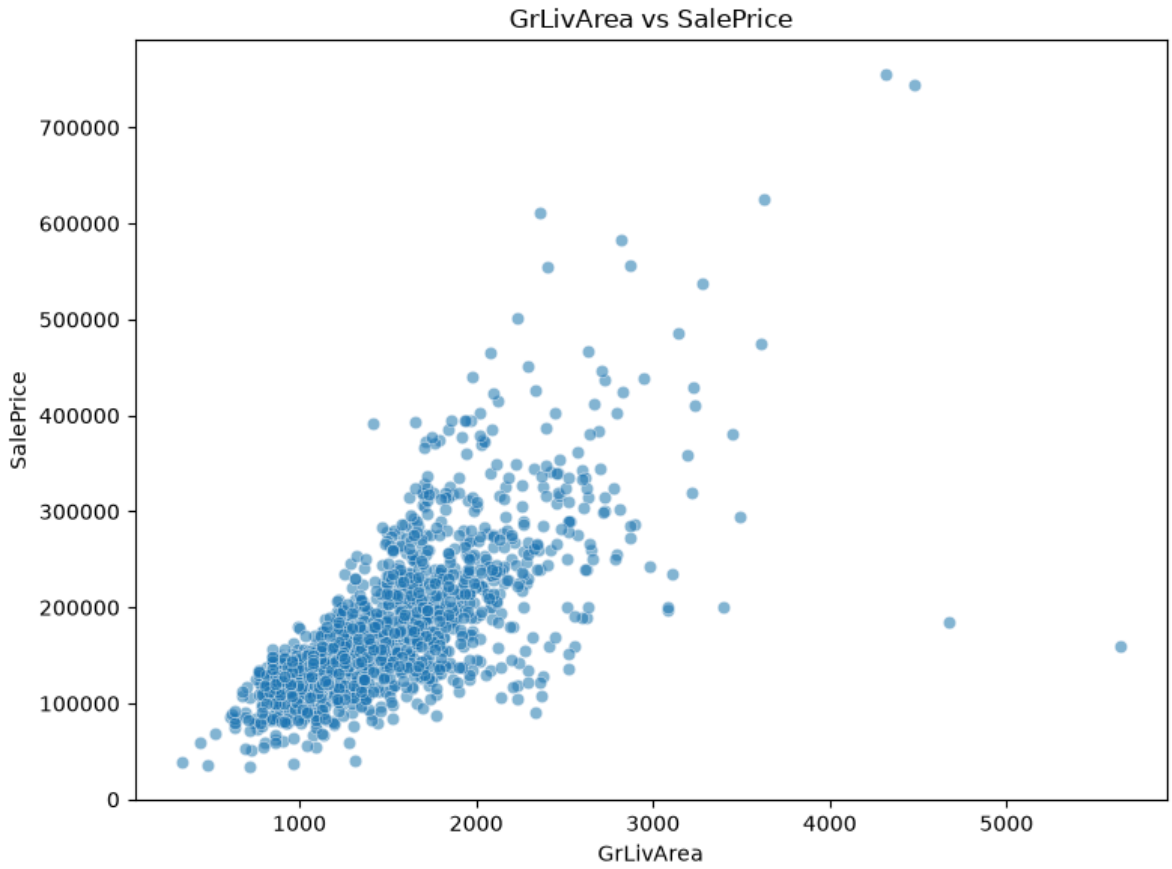
print("\nNeighborhood comparison: NAmes vs NoRidge")
for nb in ["NAmes", "NoRidge"]:
    vals = df.loc[df["Neighborhood"] == nb, "SalePrice"]
    print(f"  {nb:<8} n={len(vals):3d}  mean={money(vals.mean())}")
_, p_nb = stats.ttest_ind(
    df.loc[df["Neighborhood"] == "NAmes", "SalePrice"],
    df.loc[df["Neighborhood"] == "NoRidge", "SalePrice"],
    equal_var=False,
)
print(f"  T-test p-value: {p_nb:.2e} → {'significant' if p_nb < 0.05 e
lse 'not significant'}")

df["HomeAgeGroup"] = np.where(df["YearBuilt"] >= 2000, "New (>=2000)",
"Old (<2000)")
new_p = df.loc[df["YearBuilt"] >= 2000, "SalePrice"]
old_p = df.loc[df["YearBuilt"] < 2000, "SalePrice"]
print("\nNew vs older homes:")
print(f"  New: n={len(new_p)} ({pct(len(new_p)/len(df))}) mean={money
(new_p.mean())}")
print(f"  Old: n={len(old_p)} ({pct(len(old_p)/len(df))}) mean={money
(old_p.mean())}")
```

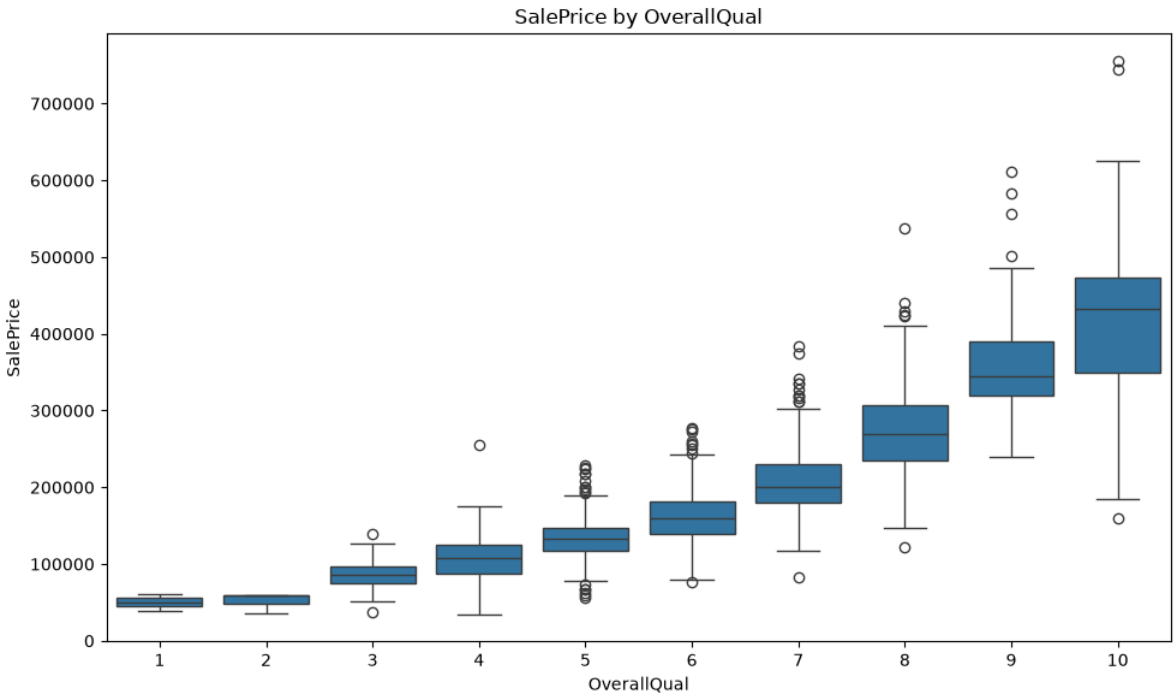
```
_, p_age = stats.ttest_ind(new_p, old_p, equal_var=False)
print(f"  T-test p-value: {p_age:.2e} → {'significant' if p_age < 0.05
else 'not significant'}")

plt.figure(figsize=(8, 6))
sns.boxplot(data=df, x="HomeAgeGroup", y="SalePrice")
plt.title("SalePrice: New vs Old Homes")
show_plot("part_c_boxplot_new_old.png")
```

```
============================================================
  Part C: Bivariate Analysis
============================================================
```

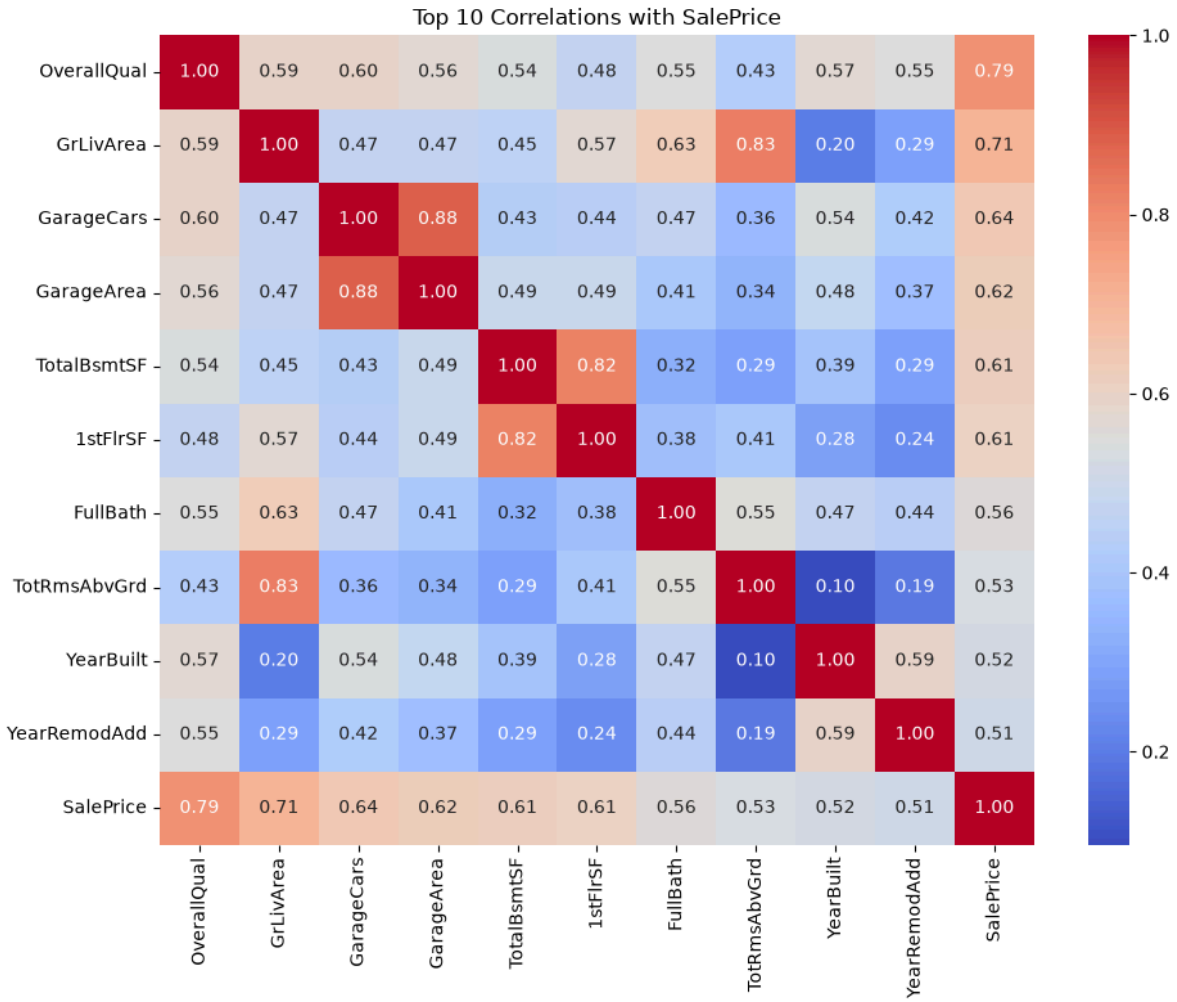



```
[saved] plots/labassignment_1/part_c_scatter.png
Trend: positive — larger GrLivArea generally means higher SalePrice.
```




```
[saved] plots/labassignment_1/part_c_boxplot_overallqual.png
```

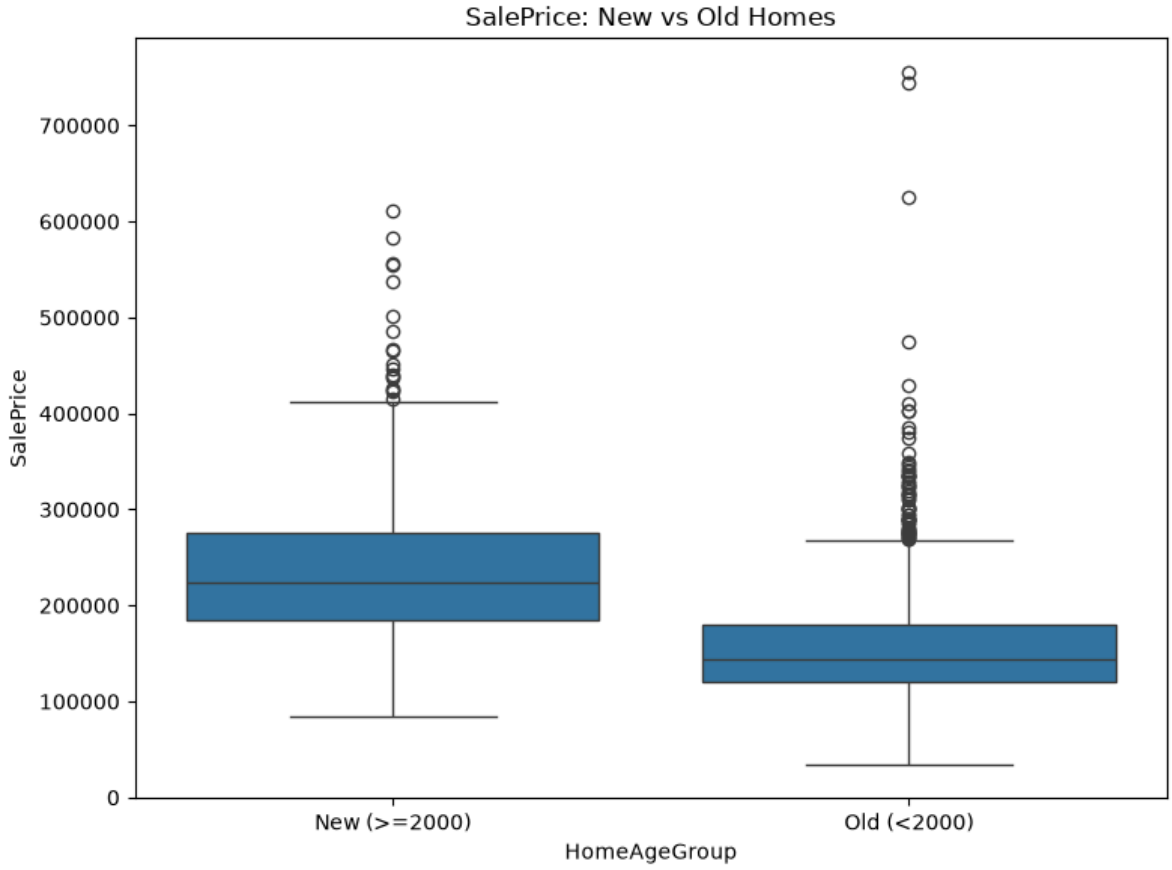
```
Top 10 features correlated with SalePrice:
Feature                    r  |r| %
-----------------------------------
OverallQual           +0.791  79.10%
GrLivArea             +0.709  70.86%
GarageCars            +0.640  64.04%
GarageArea            +0.623  62.34%
TotalBsmtSF           +0.614  61.36%
1stFlrSF              +0.606  60.59%
FullBath              +0.561  56.07%
TotRmsAbvGrd          +0.534  53.37%
YearBuilt             +0.523  52.29%
YearRemodAdd          +0.507  50.71%
```




```
[saved] plots/labassignment_1/part_c_heatmap.png

Neighborhood comparison: NAmes vs NoRidge
  NAmes    n=225  mean=$145,847.08
  NoRidge  n= 41  mean=$335,295.32
  T-test p-value: 1.79e-12 → significant

New vs older homes:
  New: n=388 (26.58%) mean=$242,439.16
  Old: n=1072 (73.42%) mean=$158,655.36
  T-test p-value: 1.10e-57 → significant
```



[saved] plots/labassignment_1/part_c_boxplot_new_old.png

Part D: Missing Value Treatment

In [5]:
```
section("Part D: Missing Value Treatment")

missing = df.isnull().sum().sort_values(ascending=False)
missing = missing[missing > 0]
miss_tbl = pd.DataFrame({
    "Missing count": missing,
    "% of rows": (100 * missing / len(df)).round(2),
})
print("Top 5 columns with most missing values:")
print(miss_tbl.head(5).to_string())

df_clean = df.copy()

# LotFrontage first: neighborhood median
df_clean["LotFrontage"] = df_clean.groupby("Neighborhood")["LotFrontag
e"].transform(
    lambda x: x.fillna(x.median())
)
df_clean["LotFrontage"] = df_clean["LotFrontage"].fillna(df_clean["Lot
Frontage"].median())

num_cols = df_clean.select_dtypes(include=[np.number]).columns
cat_cols = df_clean.select_dtypes(include=["object"]).columns

for col in num_cols:
    if df_clean[col].isnull().any():
        df_clean[col] = df_clean[col].fillna(df_clean[col].median())

for col in cat_cols:
    if df_clean[col].isnull().any():
        mode = df_clean[col].mode(dropna=True)
        fill = mode.iloc[0] if not mode.empty else "Missing"
        df_clean[col] = df_clean[col].fillna(fill)

print(f"\nRemaining missing values: {int(df_clean.isnull().sum().sum
())}")
```

```
============================================================
  Part D: Missing Value Treatment
============================================================
Top 5 columns with most missing values:
             Missing count  % of rows
PoolQC                1453    99.5200
MiscFeature           1406    96.3000
Alley                 1369    93.7700
Fence                 1179    80.7500
MasVnrType             872    59.7300

Remaining missing values: 0
```

Part E: Outlier Detection & Treatment

In [6]:
```
section("Part E: Outlier Detection (IQR)")

outlier_info = {}
for col in ["LotArea", "GrLivArea"]:
    q1, q3 = df_clean[col].quantile(0.25), df_clean[col].quantile(0.7
5)
    iqr = q3 - q1
    lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    n_out = int(((df_clean[col] < lo) | (df_clean[col] > hi)).sum())
    outlier_info[col] = {"lo": lo, "hi": hi, "n": n_out}
    print(f"{col}")
    print(f"  Bounds:   [{lo:,.2f}, {hi:,.2f}]")
    print(f"  Outliers: {n_out} ({pct(n_out / len(df_clean))})")

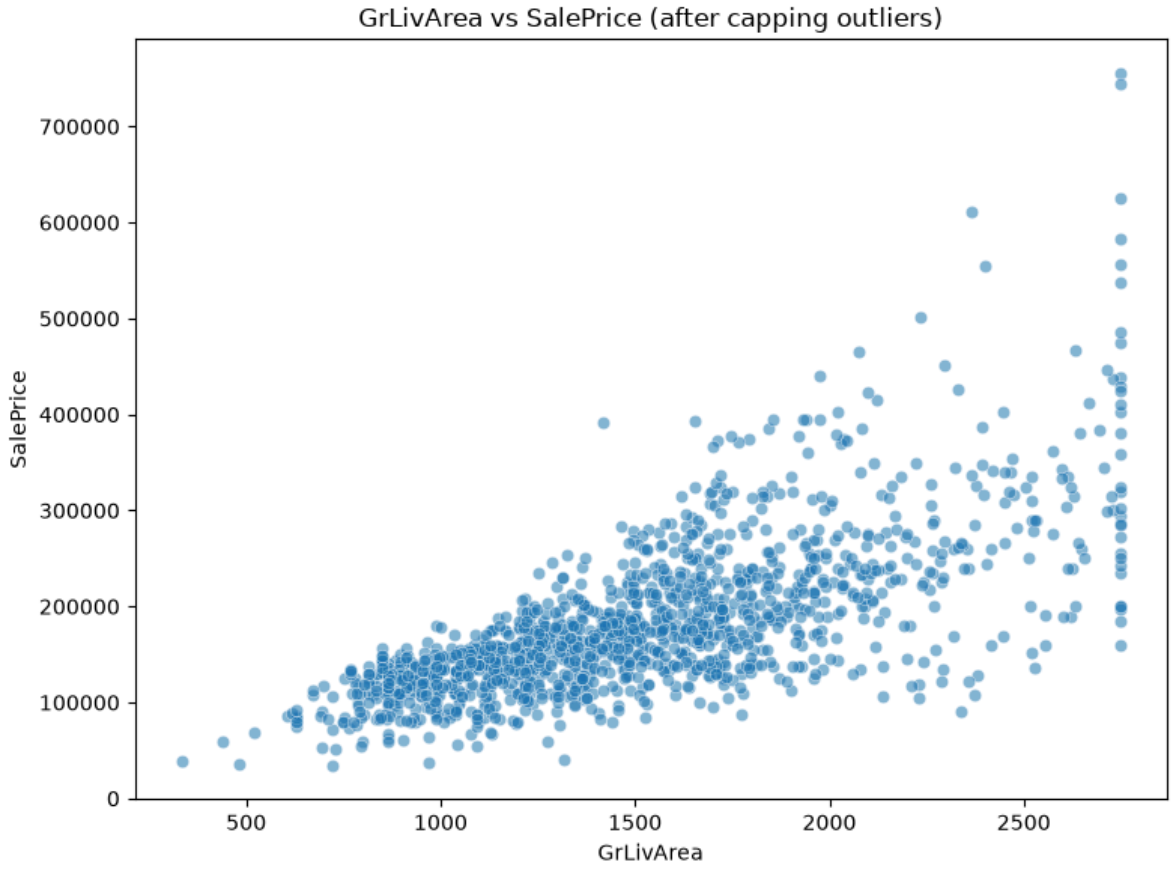
print("\nMethod: CAP outliers to IQR bounds (winsorization).")
print("Why: retains all samples while reducing extreme leverage on the
model.")

df_out = df_clean.copy()
for col in ["LotArea", "GrLivArea"]:
    df_out[col] = df_out[col].clip(outlier_info[col]["lo"], outlier_in
fo[col]["hi"])

plt.figure(figsize=(8, 6))
sns.scatterplot(data=df_out, x="GrLivArea", y="SalePrice", alpha=0.55)
plt.title("GrLivArea vs SalePrice (after capping outliers)")
show_plot("part_e_scatter_capped.png")
print("Relationship is clearer after capping — fewer extreme living-ar
ea points.")
```

```
============================================================
  Part E: Outlier Detection (IQR)
============================================================
LotArea
  Bounds:   [1,481.50, 17,673.50]
  Outliers: 69 (4.73%)
GrLivArea
  Bounds:   [158.62, 2,747.62]
  Outliers: 31 (2.12%)

Method: CAP outliers to IQR bounds (winsorization).
Why: retains all samples while reducing extreme leverage on the model.
```




```
[saved] plots/labassignment_1/part_e_scatter_capped.png
Relationship is clearer after capping — fewer extreme living-area poin
ts.
```

Part F: Feature Encoding & Scaling

- Categorical → One-Hot Encoding (low cardinality) / Label Encoding (high cardinality)
- Numerical → StandardScaler
- Save → `outputs/house_prices_clean.csv`

In [7]:

```
section("Part F: Feature Encoding & Scaling")

df_model = df_out.drop(columns=["Id", "HomeAgeGroup"], errors="ignor
e").copy()

y = df_model["SalePrice"]
X = df_model.drop(columns=["SalePrice"])

num_features = X.select_dtypes(include=[np.number]).columns.tolist()
cat_features = X.select_dtypes(include=["object"]).columns.tolist()

# Label-encode high-cardinality categoricals; one-hot the rest
high_card = [c for c in cat_features if X[c].nunique() > 10]
low_card = [c for c in cat_features if c not in high_card]

print(f"Numerical features:              {len(num_features)}")
print(f"Categorical (label encode):      {len(high_card)} → {high_car
d}")
print(f"Categorical (one-hot encode):    {len(low_card)}")

X_enc = X.copy()
label_encoders = {}
for col in high_card:
    le = LabelEncoder()
    X_enc[col] = le.fit_transform(X_enc[col].astype(str))
    label_encoders[col] = le

X_enc = pd.get_dummies(X_enc, columns=low_card, drop_first=True)
print(f"\nShape after encoding: {X_enc.shape[0]:,} rows × {X_enc.shape
[1]} columns")

# Scale numerical columns (original numeric + label-encoded)
scale_cols = num_features + high_card
scaler = StandardScaler()
X_enc[scale_cols] = scaler.fit_transform(X_enc[scale_cols])

# Combine with target for saving
clean_df = X_enc.copy()
clean_df["SalePrice"] = y.values

out_path = OUTPUTS / "house_prices_clean.csv"
clean_df.to_csv(out_path, index=False)
print(f"Scaler: StandardScaler")
print(f"Saved:  {out_path.relative_to(ROOT)}")
print(f"Final shape: {clean_df.shape[0]:,} × {clean_df.shape[1]}")
display(clean_df.head())
```

```
============================================================
  Part F: Feature Encoding & Scaling
============================================================
Numerical features:              35
Categorical (label encode):      4 → ['MSSubClass', 'Neighborhood', 'E
xterior1st', 'Exterior2nd']
Categorical (one-hot encode):    40

Shape after encoding: 1,460 rows × 194 columns
Scaler: StandardScaler
Saved:  outputs/house_prices_clean.csv
Final shape: 1,460 × 195
```

	MSSubClass	LotFrontage	LotArea	Neighborhood	OverallQual	OverallCond	YearBuilt	YearR
0	0.7864	-0.2319	-0.3332	-1.2062	0.6515	-0.5172	1.0510	
1	-0.6252	0.4370	-0.0132	1.9543	-0.0718	2.1796	0.1567	
2	0.7864	-0.0981	0.4460	-1.2062	0.6515	-0.5172	0.9848	
3	1.0687	-0.4549	-0.0271	-1.0399	0.6515	-0.5172	-1.8636	
4	0.7864	0.6154	1.2837	0.4572	1.3748	-0.5172	0.9516	

5 rows × 195 columns

Part G: Linear Regression

In [8]:
```
section("Part G: Linear Regression")

# Use cleaned encoded/scaled features from Part F
X_final = clean_df.drop(columns=["SalePrice"])
y_final = clean_df["SalePrice"]

X_train, X_test, y_train, y_test = train_test_split(
    X_final, y_final, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

train_r2 = r2_score(y_train, y_train_pred)
test_r2 = r2_score(y_test, y_test_pred)
train_rmse = np.sqrt(mean_squared_error(y_train, y_train_pred))
test_rmse = np.sqrt(mean_squared_error(y_test, y_test_pred))
train_mse = mean_squared_error(y_train, y_train_pred)
test_mse = mean_squared_error(y_test, y_test_pred)

print(f"Train size: {len(X_train):,} | Test size: {len(X_test):,}")
print(f"Features used: {X_final.shape[1]}")

print(f"\n{'Metric':<12} {'Train':>14} {'Test':>14}")
print("-" * 42)
print(f"{'R²':<12} {train_r2:>14.4f} {test_r2:>14.4f}")
print(f"{'R² %':<12} {pct(train_r2):>14} {pct(test_r2):>14}")
print(f"{'MSE':<12} {train_mse:>14,.0f} {test_mse:>14,.0f}")
print(f"{'RMSE':<12} {money(train_rmse):>14} {money(test_rmse):>14}")

# Top coefficients by absolute value
coef_df = pd.DataFrame({"Feature": X_final.columns, "Coefficient": mod
el.coef_})
coef_df["AbsCoef"] = coef_df["Coefficient"].abs()
top_coef = coef_df.sort_values("AbsCoef", ascending=False).head(10)

print("\nTop 10 coefficients by magnitude:")
print(f"{'Feature':<30} {'Coefficient':>14}")
print("-" * 46)
print(f"{'(Intercept)':<30} {model.intercept_:>14,.2f}")
for _, row in top_coef.iterrows():
    print(f"{row['Feature']:<30} {row['Coefficient']:>14,.2f}")

# Residual plot
residuals = y_test - y_test_pred
plt.figure(figsize=(8, 5))
sns.scatterplot(x=y_test_pred, y=residuals, alpha=0.55)
plt.axhline(0, color="red", linestyle="--")
plt.xlabel("Predicted SalePrice")
```

```
plt.ylabel("Residuals")
plt.title("Residual Plot — Linear Regression")
show_plot("part_g_residuals.png")

# Predicted vs Actual
plt.figure(figsize=(7, 7))
sns.scatterplot(x=y_test, y=y_test_pred, alpha=0.55)
lims = [min(y_test.min(), y_test_pred.min()), max(y_test.max(), y_test
_pred.max())]
plt.plot(lims, lims, "r--", label="Perfect prediction")
plt.xlabel("Actual SalePrice")
plt.ylabel("Predicted SalePrice")
plt.title("Actual vs Predicted SalePrice")
plt.legend()
show_plot("part_g_actual_vs_predicted.png")

print(f"\nSummary: Linear regression explains {pct(test_r2)} of SalePr
ice variance on the test set.")
print(f"Typical prediction error (RMSE): {money(test_rmse)}")
```

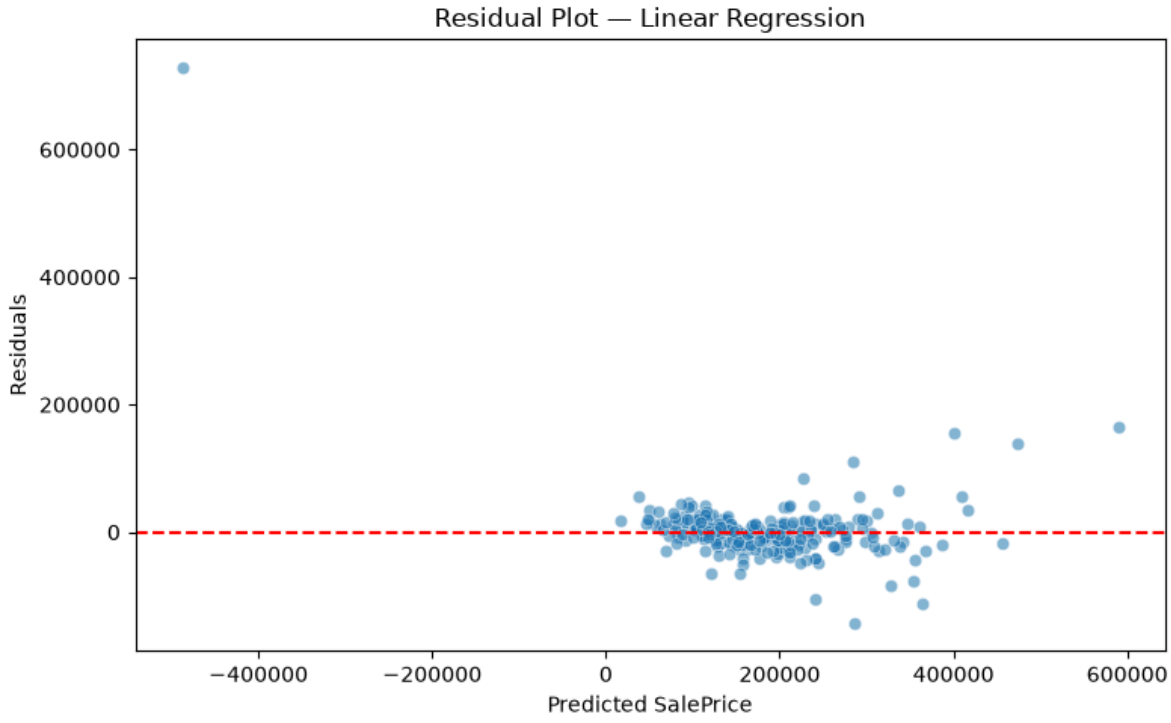
```
============================================================
  Part G: Linear Regression
============================================================

Train size: 1,168 | Test size: 292
Features used: 194
```

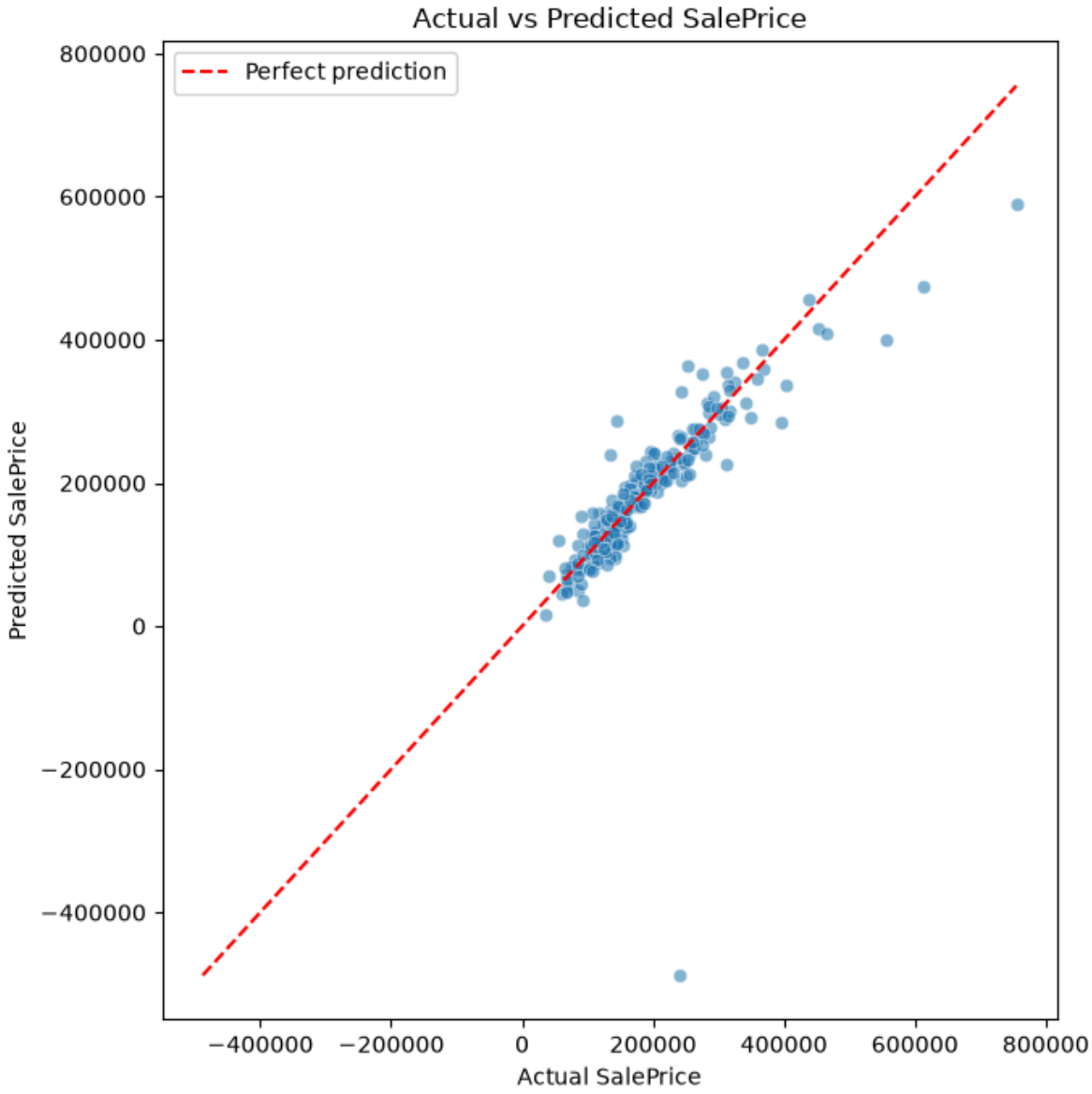
Metric	Train	Test
R^2	0.9215	0.6436
R^2 %	92.15%	64.36%
MSE	468,368,665	2,734,012,354
RMSE	$21,641.83	$52,287.78

Top 10 coefficients by magnitude:

Feature	Coefficient
(Intercept)	−500,541.54
RoofMatl_WdShngl	733,861.44
RoofMatl_CompShg	718,590.56
RoofMatl_Roll	709,743.80
RoofMatl_WdShake	707,049.90
RoofMatl_Tar&Grv	697,964.86
RoofMatl_Metal	695,514.40
Condition2_PosN	−260,543.42
GarageQual_Fa	−148,245.44
GarageQual_Po	−147,933.26
GarageQual_TA	−144,762.89




```
[saved] plots/labassignment_1/part_g_residuals.png
```




```
[saved] plots/labassignment_1/part_g_actual_vs_predicted.png

Summary: Linear regression explains 64.36% of SalePrice variance on th
e test set.
Typical prediction error (RMSE): $52,287.78
```